Лабораторная работа 3

Бавтута Дмитрий Валерьевич

6203-010302D

Task 2

Реализация классов исключений

```java
package functions;

You, 2 недели назад | 1 author (You)
public class FunctionPointIndexOutOfBoundsException extends IndexOutOfBoundsException {

    public FunctionPointIndexOutOfBoundsException() {
        super(s: "Индекс точки функции выходит за границы");
    }

    public FunctionPointIndexOutOfBoundsException(String message) {
        super(message);
    }

    public FunctionPointIndexOutOfBoundsException(int index) {
        super("Индекс точки функции выходит за границы: " + index);
    }
}
```

```java
package functions;

public class InappropriateFunctionPointException extends RuntimeException {

    public InappropriateFunctionPointException() {
        super(message: "Некорректная точка функции");
    }

    public InappropriateFunctionPointException(String message) {
        super(message);
    }

    public InappropriateFunctionPointException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

Task 3

Изменил сообщения об исключениях во всех методах и конструкторах класса TabulatedFunction (ArrayTabulatedFunction)

Ниже примеры реализации(Не все)

```java
public ArrayTabulatedFunction(double leftX, double rightX, int pointsCount) {
    if (leftX >= rightX - EPSILON) {
        throw new IllegalArgumentException(s: "Левая граница должна быть меньше правой границы");
    }
    if (pointsCount < 2) {
        throw new IllegalArgumentException(s: "Количество точек должно быть не менее 2");
    }
```

```java
public ArrayTabulatedFunction(double leftX, double rightX, double[] values) {
    if (leftX >= rightX - EPSILON) {
        throw new IllegalArgumentException(s: "Левая граница должна быть меньше правой границы");
    }
    if (values.length < 2) {
        throw new IllegalArgumentException(s: "Количество точек должно быть не менее 2");
    }
```

```java
@Override
public FunctionPoint getPoint(int index) {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(index);
    }
    return new FunctionPoint(points[index]);
}
```

```java
@Override
public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(index);
    }
```

```java
private void setFirstPoint(FunctionPoint point) throws InappropriateFunctionPointException {
    if (point.getX() >= points[1].getX() - EPSILON) {
        throw new InappropriateFunctionPointException(
            message: "X координата первой точки должна быть меньше X координаты второй точки");
    }
    points[0] = new FunctionPoint(point);
```

```java
@Override
public void deletePoint(int index) {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(index);
    }
    if (pointsCount < 3) {
        throw new IllegalStateException(s: "Нельзя удалить точку - функция должна содержать минимум 3 точки");
    }
```

Task 4

Реализация класса связного циклического списка LinkedListTabulatedFunction и внутреннего класса FunctionNode для описания узла списка который хранит данные о самой точке и о следующем и предыдущем элементах.

```java
package functions;

public class LinkedListTabulatedFunction implements TabulatedFunction {
    private static final double EPSILON = 1e-10;

    private FunctionNode head;
    private FunctionNode lastAccessedNode;
    private int lastAccessedIndex;
    private int pointsCount;
```

```java
private static class FunctionNode {
    private FunctionPoint point;
    private FunctionNode prev;
    private FunctionNode next;
```

```
private FunctionNode getNodeByIndex(int index) {
        if (index < 0 || index >= pointsCount) {
            throw new FunctionPointIndexOutOfBoundsException(index);
        }

        // Оптимизация: выбор оптимального пути доступа
        FunctionNode currentNode;
        int currentIndex;

        if (lastAccessedNode != head && lastAccessedIndex != -1) {
            int distanceFromLast = Math.abs(index - lastAccessedIndex);
            int distanceFromStart = index;
            int distanceFromEnd = pointsCount - 1 - index;

            if (distanceFromLast <= distanceFromStart && distanceFromLast <= distanceFromEnd) {
                currentNode = lastAccessedNode;
                currentIndex = lastAccessedIndex;

                if (index > lastAccessedIndex) {
                    while (currentIndex < index) {
                        currentNode = currentNode.getNext();
                        currentIndex++;
                    }
                } else {
                    while (currentIndex > index) {
                        currentNode = currentNode.getPrev();
                        currentIndex--;
                    }
                }
            } else if (distanceFromStart <= distanceFromEnd) {
                currentNode = head.getNext();
                currentIndex = 0;
                while (currentIndex < index) {
                    currentNode = currentNode.getNext();
                    currentIndex++;
                }
            } else {
                currentNode = head.getPrev();
                currentIndex = pointsCount - 1;
                while (currentIndex > index) {
                    currentNode = currentNode.getPrev();
                    currentIndex--;
                }
            }
        } else {
            currentNode = head.getNext();
            currentIndex = 0;
            while (currentIndex < index) {
                currentNode = currentNode.getNext();
                currentIndex++;
            }
        }

        // Кэшируем последний доступ
        lastAccessedNode = currentNode;
        lastAccessedIndex = index;

        return currentNode;
    }
```

```
private FunctionNode addNodeToTail() {
    FunctionNode newNode = new FunctionNode(new FunctionPoint(x: 0, y: 0));
    insertNodeAtTail(newNode);
    return newNode;
}
```

```java
// Вспомогательный метод для вставки узла в конец
private void insertNodeAtTail(FunctionNode newNode) {
    if (pointsCount == 0) {
        newNode.setPrev(head);
        newNode.setNext(head);
        head.setNext(newNode);
        head.setPrev(newNode);
    } else {
        FunctionNode lastNode = head.getPrev();
        newNode.setPrev(lastNode);
        newNode.setNext(head);
        lastNode.setNext(newNode);
        head.setPrev(newNode);
    }

    pointsCount++;
    lastAccessedNode = newNode;
    lastAccessedIndex = pointsCount - 1;
}
```

```java
// 5. Метод добавления узла по индексу
private FunctionNode addNodeByIndex(int index) {
    if (index < 0 || index > pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(index);
    }

    if (index == pointsCount) {
        return addNodeToTail();
    }

    FunctionNode currentNode = getNodeByIndex(index);
    FunctionNode newNode = new FunctionNode(new FunctionPoint(x: 0, y: 0));
    FunctionNode prevNode = currentNode.getPrev();

    // Вставляем новый узел перед currentNode
    newNode.setPrev(prevNode);
    newNode.setNext(currentNode);
    prevNode.setNext(newNode);
    currentNode.setPrev(newNode);

    pointsCount++;
    lastAccessedNode = newNode;
    lastAccessedIndex = index;

    return newNode;
}
```

```java
// 6. Метод удаления узла по индексу
private FunctionNode deleteNodeByIndex(int index) {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(index);
    }
    if (pointsCount < 3) {
        throw new IllegalStateException(s: "Нельзя удалить точку - функция должна содержать минимум 3 точки");
    }

    FunctionNode nodeToDelete = getNodeByIndex(index);
    FunctionNode prevNode = nodeToDelete.getPrev();
    FunctionNode nextNode = nodeToDelete.getNext();

    // Перелинковываем соседние узлы
    prevNode.setNext(nextNode);
    nextNode.setPrev(prevNode);

    pointsCount--;

    // Обновляем кэш
    if (lastAccessedNode == nodeToDelete) {
        lastAccessedNode = head;
        lastAccessedIndex = -1;
    } else if (lastAccessedIndex > index) {
        lastAccessedIndex--;
    }

    return nodeToDelete;
}
```

Task 5

```java
public LinkedListTabulatedFunction(double leftX, double rightX, int pointsCount) {
    if (leftX >= rightX - EPSILON) {
        throw new IllegalArgumentException(s: "Левая граница должна быть меньше правой границы");
    }
    if (pointsCount < 2) {
        throw new IllegalArgumentException(s: "Количество точек должно быть не менее 2");
    }

    initializeHead();

    double step = (rightX - leftX) / (pointsCount - 1);
    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step;
        // Создаем узел с точкой сразу
        FunctionNode newNode = new FunctionNode(new FunctionPoint(x, y: 0.0));
        // Вставляем в конец
        insertNodeAtTail(newNode);
    }
}
```

```java
public ArrayTabulatedFunction(double leftX, double rightX, int pointsCount) {
    if (leftX >= rightX - EPSILON) {
        throw new IllegalArgumentException(s: "Левая граница должна быть меньше правой границы");
    }
    if (pointsCount < 2) {
        throw new IllegalArgumentException(s: "Количество точек должно быть не менее 2");
    }

    this.pointsCount = pointsCount;
    this.points = new FunctionPoint[pointsCount];

    double step = (rightX - leftX) / (pointsCount - 1);

    for(int i = 0; i < pointsCount; ++i) {
        double x = leftX + i * step;
        points[i] = new FunctionPoint(x, y: 0.0);
    }
}
```

```java
@Override
public double getPointY(int index) {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(index);
    }
    FunctionNode node = getNodeByIndex(index);
    return node.getPoint().getY();
}

@Override
public void setPointY(int index, double y) {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(index);
    }
    FunctionNode node = getNodeByIndex(index);
    node.getPoint().setY(y);
}
```

```java
@Override
public double getPointY(int index) {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(index);
    }
    return points[index].getY();
}

@Override
public void setPointY(int index, double y) {
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException(index);
    }
    points[index].setY(y);
}
```

Task 6

```java
package functions;

public interface TabulatedFunction {

    // Методы доступа к области определения
    double getLeftDomainBorder();
    double getRightDomainBorder();

    // Методы работы с функцией
    double getFunctionValue(double x);
    int getPointsCount();

    // Методы работы с точками
    FunctionPoint getPoint(int index);
    void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException;

    // Методы работы с координатами точек
    double getPointX(int index);
    void setPointX(int index, double x) throws InappropriateFunctionPointException;
    double getPointY(int index);
    void setPointY(int index, double y);

    // Методы модификации набора точек
    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException;
    void deletePoint(int index);
}
```

Task 7

```java
import functions.*;

public class Main {

    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        System.out.println(x: " ТЕСТИРОВАНИЕ КЛАССОВ ТАБУЛИРОВАННЫХ ФУНКЦИЙ \n");

        // Тестируем ArrayTabulatedFunction
        System.out.println(x: "1. ТЕСТИРОВАНИЕ ArrayTabulatedFunction:");
        TabulatedFunction arrayFunc = new ArrayTabulatedFunction(leftX: 0, rightX: 10, pointsCount: 5);
        testTabulatedFunction(arrayFunc);

        // Тестируем LinkedListTabulatedFunction
        System.out.println(x: "\n2. ТЕСТИРОВАНИЕ LinkedListTabulatedFunction:");
        TabulatedFunction linkedListFunc = new LinkedListTabulatedFunction(leftX: 0, rightX: 10, pointsCount: 5);
        testTabulatedFunction(linkedListFunc);

        // Тестируем исключения конструкторов
        System.out.println(x: "\n3. ТЕСТИРОВАНИЕ ИСКЛЮЧЕНИЙ В КОНСТРУКТОРАХ:");
        testConstructorExceptions();

        System.out.println(x: "\n ВСЕ ТЕСТЫ ЗАВЕРШЕНЫ ");
    }
```

Вывод класса Main

ТЕСТИРОВАНИЕ КЛАССОВ ТАБУЛИРОВАННЫХ ФУНКЦИЙ

1. ТЕСТИРОВАНИЕ ArrayTabulatedFunction:

Исходная функция: ArrayTabulatedFunction[(0.0, 0.0), (2.5, 0.0), (5.0, 0.0), (7.5, 0.0), (10.0, 0.0)]

Количество точек: 5

Тест исключений для некорректных индексов

getPoint(-1): Индекс точки функции выходит за границы: -1

getPoint(100): Индекс точки функции выходит за границы: 100

getPointX(-5): Индекс точки функции выходит за границы: -5

setPointY(10, 5.0): Индекс точки функции выходит за границы: 10

Тест исключений для некорректных точек

setPoint(0) с нарушением порядка: X координата первой точки должна быть меньше X координаты второй точки

setPoint(last) с нарушением порядка: X координата последней точки должна быть больше X координаты предыдущей точки

setPoint(2) с нарушением диапазона: X координата средней точки должна быть между 2.5 и 7.5

setPointX(0, 5.0) с нарушением порядка: X координата первой точки должна быть меньше X координаты второй точки

Тест исключений при добавлении точек

addPoint с существующим X: Точка с X=2.5 уже существует

addPoint с новым X: успешно добавлена

Теперь точек: 6

Тест исключений при удалении точек

deletePoint(100): Индекс точки функции выходит за границы: 100

deletePoint(-1): Индекс точки функции выходит за границы: -1

Создана минимальная функция для теста удаления

deletePoint при pointsCount < 3: Нельзя удалить точку - функция должна содержать минимум 3 точки

Тест корректных операций

Левая граница: 0.0

Правая граница: 10.0

координата точки 1: 0.0

координата точки 1 после setPointY: 15.0

Точка 2: (3.70, 7.50)

f(2.5) = 15.0

f(5.0) = 0.0

f(15.0) = NaN

f(-5.0) = NaN

координата точки 4 после корректного изменения: 7.6

Добавлена новая точка, теперь точек: 7

Удалена точка 1, теперь точек: 6

Все корректные операции выполнены успешно

2. ТЕСТИРОВАНИЕ LinkedListTabulatedFunction:

Исходная функция: LinkedListTabulatedFunction[(0.0, 0.0), (2.5, 0.0), (5.0, 0.0), (7.5, 0.0), (10.0, 0.0)]

Количество точек: 5

Тест исключений для некорректных индексов

getPoint(-1): Индекс точки функции выходит за границы: -1

getPoint(100): Индекс точки функции выходит за границы: 100

getPointX(-5): Индекс точки функции выходит за границы: -5

setPointY(10, 5.0): Индекс точки функции выходит за границы: 10

Тест исключений для некорректных точек

setPoint(0) с нарушением порядка: X координата первой точки должна быть меньше X координаты второй точки

setPoint(last) с нарушением порядка: X координата последней точки должна быть больше X координаты предыдущей точки

setPoint(2) с нарушением диапазона: X координата средней точки должна быть между 2.5 и 7.5

setPointX(0, 5.0) с нарушением порядка: X координата первой точки должна быть меньше X координаты второй точки

Тест исключений при добавлении точек

addPoint с существующим X: Точка с X=2.5 уже существует

addPoint с новым X: успешно добавлена

Теперь точек: 6

Тест исключений при удалении точек

deletePoint(100): Индекс точки функции выходит за границы: 100

deletePoint(-1): Индекс точки функции выходит за границы: -1

Создана минимальная функция для теста удаления

deletePoint при pointsCount < 3: Нельзя удалить точку - функция должна содержать минимум 3 точки

Тест корректных операций

Левая граница: 0.0

Правая граница: 10.0

координата точки 1: 0.0

координата точки 1 после setPointY: 15.0

Точка 2: (3.70, 7.50)

f(2.5) = 15.0

f(5.0) = 0.0

f(15.0) = NaN

f(-5.0) = NaN

координата точки 4 после корректного изменения: 7.6

Удалена точка 1, теперь точек: 5

Все корректные операции выполнены успешно

3. ТЕСТИРОВАНИЕ ИСКЛЮЧЕНИЙ В КОНСТРУКТОРАХ:

Тест исключений конструкторов ArrayTabulatedFunction

ArrayTabulatedFunction(10, 5, 5): Левая граница должна быть меньше правой границы

ArrayTabulatedFunction(0, 10, 1): Количество точек должно быть не менее 2

ArrayTabulatedFunction(10, 5, [...]): Левая граница должна быть меньше правой границы

ArrayTabulatedFunction(0, 10, [один элемент]): Количество точек должно быть не менее 2

Тест исключений конструкторов LinkedListTabulatedFunction

LinkedListTabulatedFunction(10, 5, 5): Левая граница должна быть меньше правой границы

LinkedListTabulatedFunction(0, 10, 1): Количество точек должно быть не менее 2

LinkedListTabulatedFunction(10, 5, [...): Левая граница должна быть меньше правой границы

LinkedListTabulatedFunction(0, 10, [один элемент]): Количество точек должно быть не менее 2

ВСЕ ТЕСТЫ ЗАВЕРШЕНЫ